

**ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**PHÁT TRIỂN ỨNG DỤNG
TRÊN THIẾT BỊ DI ĐỘNG (CO3043)**

**BÁO CÁO BÀI TẬP LỚN
ỨNG DỤNG CHIA SẺ TÀI LIỆU
BK_SHARING**

GVHD: Hoàng Lê Hải Thanh

Nhóm: L02_Tứ ca

STT	Họ Và Tên	MSSV	Lớp	Tỉ lệ hoàn thành
1	Đặng Minh Quang	2312785	L02	100%
2	Nguyễn Duy Thông	2213331	L02	100%
3	Hà Khởi Nguyên	2212287	L02	100%
4	Hoàng Giữ Tiến Nhất	2312460	L02	100%

THÀNH PHỐ HỒ CHÍ MINH, 2025



Mục lục

I. Giới thiệu

1. Vấn đề và Bối cảnh

Trong môi trường học tập hiện nay, sinh viên thường gặp khó khăn khi tìm kiếm tài liệu học tập chất lượng và tập trung. Nhiều tài liệu phân tán trên nhiều nguồn như qua các nền tảng mạng xã hội (Facebook, Zalo,..) và những vấn đề phổ biến khi sinh viên tìm tài liệu có thể gặp như là:

- **Thất lạc thông tin:** Tài liệu dễ bị trôi bài, khó tìm kiếm lại sau một thời gian.
- **Thiếu tính hệ thống:** Không có sự phân loại rõ ràng theo môn học, khoa hay học kỳ.
- **Độ tin cậy:** Khó kiểm chứng được chất lượng và nguồn gốc của tài liệu.

2. Mục tiêu và Tầm nhìn

Dự án **BK Sharing** ra đời với mục tiêu xây dựng một nền tảng di động tập trung, chuyên nghiệp và an toàn để:

- Cung cấp kho lưu trữ tài liệu học thuật số hóa chất lượng cao.
- Tối ưu hóa thời gian tìm kiếm nguồn học liệu uy tín cho sinh viên.
- Xây dựng cộng đồng chia sẻ tri thức tự nguyện và bền vững.
- Cho phép người dùng đăng ký/đăng nhập và quản lý tài khoản.
- Cho phép người dùng tải lên tài liệu (PDF, DOCX, PPT, ...), thêm mô tả và gắn tag.
- Hỗ trợ duyệt, tìm kiếm và tải tài liệu.
- Định hướng phát triển dự án: Là nền tảng để mở rộng cho tính năng bình luận, xếp hạng, chat nhóm, và AI gợi ý

Ứng dụng hướng tới sinh viên đại học, giảng viên và cộng đồng học tập trực tuyến.

3. Mục đích và Tác động

Mục đích của ứng dụng là tạo ra một ”thư viện bỏ túi” cho mỗi sinh viên. Tác động của dự án không chỉ dừng lại ở việc cung cấp file, mà còn thúc đẩy văn hóa chia sẻ, hỗ trợ lẫn nhau trong học tập, góp phần nâng cao kết quả học tập chung của cộng đồng sinh viên.

II. Phương pháp và Quy trình phát triển

1. Quy trình phát triển

Nhóm áp dụng mô hình Agile/Scrum để đảm bảo tính linh hoạt. Dự án được chia thành các sprint ngắn (1–2 tuần) để điều chỉnh theo phản hồi thực tế. Các hoạt động chính gồm:

1. Thu thập yêu cầu và xác định MVP qua Impact-Effort Matrix.
2. Thiết kế luồng người dùng và kiến trúc 3 tầng (Presentation, Application, Data).
3. Triển khai frontend React Native (Expo) và backend Spring Boot.
4. Viết API, tài liệu Postman, và triển khai trên Fly.io; cơ sở dữ liệu Supabase (PostgreSQL).
5. Kiểm thử tự động và thủ công; thu thập phản hồi người dùng.

2. Công cụ cộng tác và quản lý mã nguồn

- **Github:** pull request, code review. Quản lý phiên bản tập trung. Nhóm chia nhánh rõ ràng (frontend/code và backend/code) để làm việc song song.
- **Quản lý công việc:** Issues/Projects trên GitHub và Trello cho backlog và sprint.
- **Postman:** Sử dụng làm công cụ chính để thiết kế, thử nghiệm và viết tài liệu API.
- **CI/CD:** GitHub Actions.

3. Cơ sở ra quyết định (Decision-making Rationale)

Nhóm lựa chọn kiến trúc **3-Tier (3 lớp)** vì sự tách biệt rõ ràng giữa giao diện, logic nghiệp vụ và dữ liệu.

- **Frontend (React Native):** Phù hợp cho sinh viên vì khả năng đa nền tảng.
- **Backend (Spring Boot):** Lựa chọn vì tính bảo mật cao, hỗ trợ tốt cho Java 17 và dễ dàng tích hợp với PostgreSQL.

III. Tính năng và chức năng của ứng dụng

1. Tính năng cốt lõi - MVP

Phiên bản MVP tập trung vào các tính năng giúp người dùng thực hiện luồng công việc chính: Tìm kiếm → Tải lên → Chia sẻ. Dự án BK Sharing tập trung vào các tính năng cốt lõi như:

- **Đăng ký / Đăng nhập:** Xác thực người dùng để lưu trữ tài khoản và tài liệu cá nhân. Tính năng này giúp định danh người dùng, theo dõi hoạt động và bảo mật thông tin. Nó là nền tảng cho các chức năng khác như upload tài liệu.
- **Tải lên tài liệu:** Cho phép người dùng chia sẻ file PDF, DOCX, PPT (v.v.) kèm mô tả. Khi upload, người dùng có thể chọn môn học và gắn tag để tăng khả năng tìm kiếm. Đây là tính năng trọng tâm để thu hút nội dung cộng đồng.
- **Duyệt và tải tài liệu:** Người dùng có thể xem danh sách tài liệu do cộng đồng đăng lên theo môn học hoặc tag, sau đó tải về máy để học offline. Điều này đem lại giá trị ngay lập tức, giúp sinh viên có nguồn tài liệu phong phú.
- **Tìm kiếm tài liệu:** Cho phép tìm kiếm theo từ khóa (tên môn, tên tài liệu, tên người đăng...). Tính năng này cải thiện trải nghiệm người dùng, giúp tìm đúng tài liệu khi kho dữ liệu lớn dần.
- **Gắn thẻ theo môn học:** Hệ thống hỗ trợ tag để phân loại tài liệu theo môn học hoặc chủ đề. Việc gắn thẻ đơn giản (trong form upload) giúp tổ chức tài liệu hiệu quả và hỗ trợ việc tìm kiếm, sắp xếp nhanh chóng.
- **Hồ sơ cá nhân cơ bản:** Người dùng có thể xem thông tin cá nhân và danh sách tài liệu đã đăng tải. Cung cấp nền tảng cá nhân hóa và là bước đầu để phát triển hồ sơ học tập chi tiết trong tương lai.
- **Giao diện thân thiện và đa thiết bị:** Thiết kế UI theo chuẩn Material Design, đơn giản và trực quan. Ứng dụng tương thích với điện thoại và máy tính bảng, thanh điều hướng dưới (Bottom Tabs) gồm các mục Home, Archive, Settings, Notifications để thao tác thuận tiện.
- **Trang Giới thiệu / Hướng dẫn sử dụng:** Cung cấp thông tin ngắn gọn về mục tiêu và cách sử dụng ứng dụng. Dù tác động không lớn, nhưng trang này tăng mức độ tin cậy và hỗ trợ người dùng mới làm quen nhanh.

Ngoài ra, các tính năng nâng cao (nice-to-have) như bình luận, đánh giá tài liệu, gợi ý AI, chat nhóm... được xem xét phát triển trong tương lai khi có phản hồi người dùng và dữ liệu thực nghiệm từ phiên bản MVP.

Effort matrix MVP:

Tính năng	Impact	Effort	Ưu tiên	Lý do
Đăng ký/Đăng nhập	Cao	Cao	4 sao	Nền tảng xác thực người dùng.
Tải lên tài liệu	Cao	Cao	5 sao	Tính năng cốt lõi nhất.

Duyệt & Tải tài liệu	Cao	Cao	5 sao	Giá trị thực tế cho người dùng.
Tìm kiếm	Cao	Cao	4 sao	Nâng cao trải nghiệm tìm kiếm.
Gắn thẻ (Tagging)	Cao	Thấp	3 sao	Tổ chức tài liệu hiệu quả.

2. Tính năng mở rộng

Hồ sơ người dùng, giao diện responsive, trang giới thiệu/hướng dẫn.

3. Các tính năng nâng cao (Advanced Functionalities)

Bên cạnh MVP, hệ thống tích hợp các tính năng như:

- **Quản lý phiên bản (Versioning):** Cho phép cập nhật tài liệu mà không mất dữ liệu cũ.
- **Nhật ký hệ thống (Audit Logs):** Ghi lại các thao tác quan trọng để đảm bảo tính minh bạch và bảo mật dữ liệu.

IV. Chi tiết triển khai

1. Công nghệ sử dụng

- **Frontend:** React Native + Expo Router. Ứng dụng tuân theo cấu trúc folder-based routing: app/, app/(auth)/, app/(tabs)/, app/components/, app/context/ cho việc điều hướng mượt mà..
- **Backend:** Spring Boot (Java 17+), sử dụng Maven để quản lý thư viện và Docker để đóng gói. Hỗ trợ REST API.
- **Database:** PostgreSQL được quản lý trên nền tảng Supabase.
- **Deployment:** Fly.io cho backend; Supabase (managed DB). Frontend mobile build phân phối qua Expo.
- **API docs:** Postman collection để mô tả endpoint, request/response và mẫu dữ liệu.

2. Cấu trúc mã nguồn

Frontend : Chia thành các thư mục app/, components/, context/ để quản lý trạng thái toàn cục. Thư mục chính app/ chứa các route và layout, nhóm route (auth) cho màn hình đăng

nhập/đăng ký và (tabs) cho các màn hình chính như Home, Archive, Settings. Các component tái sử dụng lưu trong app/components/.

Backend : Kiến trúc phân lớp chuẩn (Controller - Service - Repository - Entity). Backend là Spring Boot project có cấu hình Docker Compose cho môi trường phát triển/triển khai. Các chú ý:

- Maven build: `mvn clean package -DskipTests`
- Chạy với Docker Compose: `docker-compose up -d --build`
- File cấu hình `application.properties` chứa kết nối tới PostgreSQL/Supabase.
- Seed dữ liệu và tài khoản mặc định (admin/student) được cung cấp.

V. Trải nghiệm người dùng

1. Luồng người dùng

1. Người dùng mở ứng dụng. Nếu chưa đăng nhập, chuyển tới màn hình Đăng nhập/Đăng ký.
2. Sau khi đăng nhập, người dùng được chuyển đến trang chủ (Home) hiển thị các tài liệu mới nhất.
3. Sử dụng thanh tìm kiếm để lọc tài liệu theo môn học.
4. Người dùng mở trang Chi tiết tài liệu, đọc mô tả, và có thể tải về hoặc lưu vào danh sách cá nhân.
5. Đăng tải: người dùng chọn file, điền tên, mô tả, tag, chọn môn học và bấm Upload.
6. Lưu trữ: người dùng vào trang lưu trữ để xem các tài liệu đã được lưu và các tài liệu do người dùng đã chia sẻ trên ứng dụng.
7. Thông báo: đây là trang thông báo các hành động của người dùng và các lời nhắc hệ thống.
8. Cài đặt: người dùng vào trang cài đặt để có thể chỉnh sửa hồ sơ, xem các thông tin về bảo mật và điều khoản.

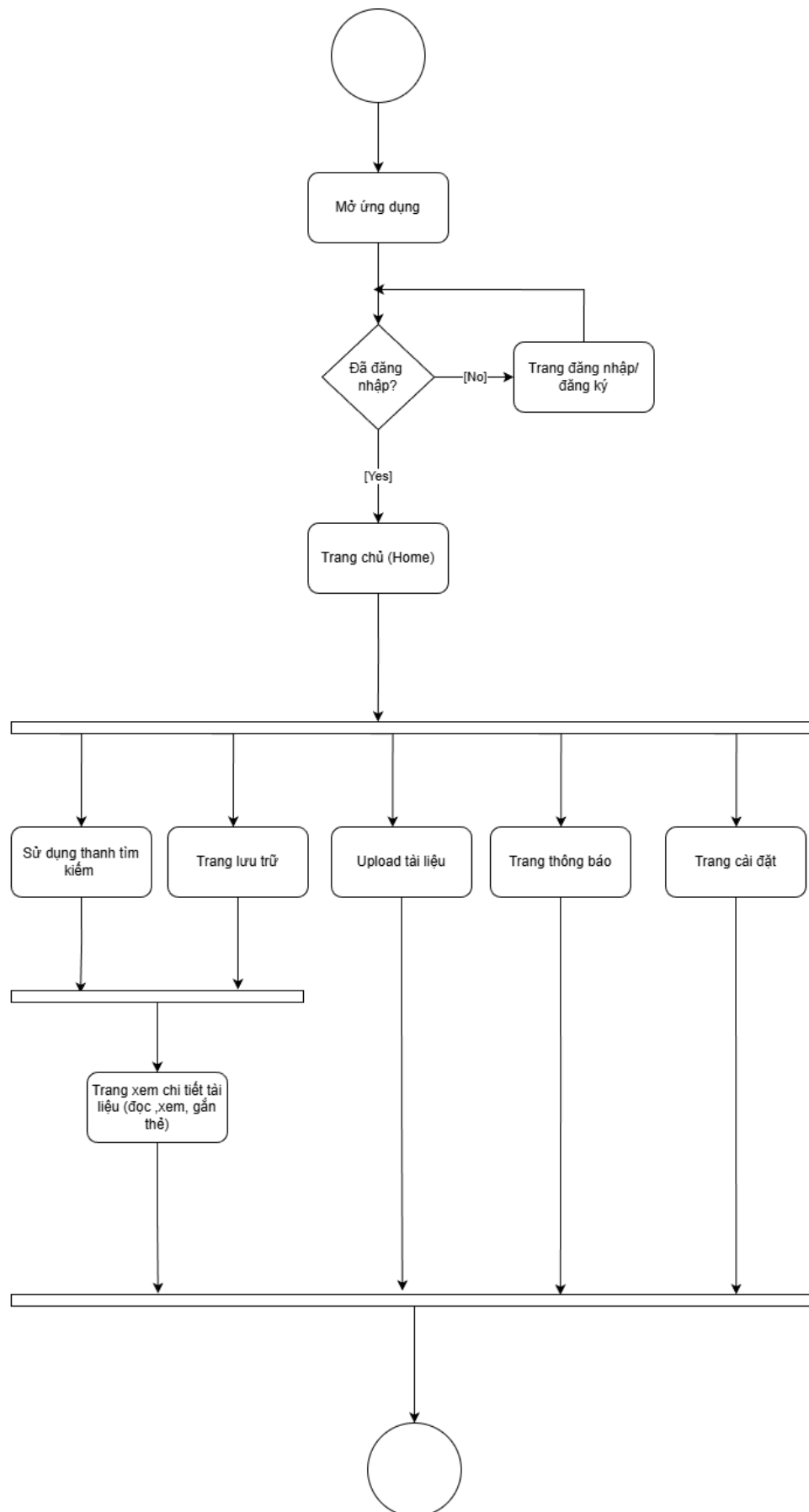
Luồng người dùng (user flow) cho MVP như sau: khi khởi động, người dùng vào màn hình **Onboarding** giới thiệu ngắn gọn. Sau đó, họ đến màn hình **Đăng ký / Đăng nhập**. Sau khi đăng nhập thành công, giao diện chính hiện ra với thanh điều hướng dưới gồm các tab *Home*, *Archive*, *Settings*, *Notifications*.

- **Trang chủ (Home):** Hiển thị danh sách tài liệu (theo xu hướng hoặc mới nhất). Có ô tìm

kiếm ở đầu màn hình để tìm nhanh theo từ khóa. Danh sách có thể gồm các nhóm môn học; chọn vào tài liệu sẽ mở màn hình chi tiết.

- **Trang lưu trữ:** Được chia thành hai phần chính là tài liệu người dùng đã tải xuống và tài liệu người dùng đã chia sẻ. Cho phép duyệt tài liệu theo danh mục, môn học, hoặc tag. Người dùng có thể lọc và sắp xếp danh sách. Mỗi mục hiển thị hình ảnh thu nhỏ (nếu có), tên tài liệu và tên người đăng.
- **Chi tiết tài liệu:** Khi chọn một tài liệu, hiển thị thông tin chi tiết như ảnh bìa, mô tả, người đăng, và nút *Download*. Tại đây người dùng cũng thấy các tag liên quan và có thể tác động (ví dụ ấn *Thích*).
- **Trang Cài đặt (Settings):** Người dùng có thể chỉnh sửa thông tin cá nhân cơ bản (tên, ảnh đại diện) và xem danh sách tài liệu đã đăng. Bên cạnh đó còn có liên kết đến trang *About/Help*.
- **Trang Thông báo (Notifications):** Hiển thị các thông báo hệ thống hoặc từ cộng đồng (ví dụ: tài liệu mới từ tag đang theo dõi, phản hồi từ người khác).

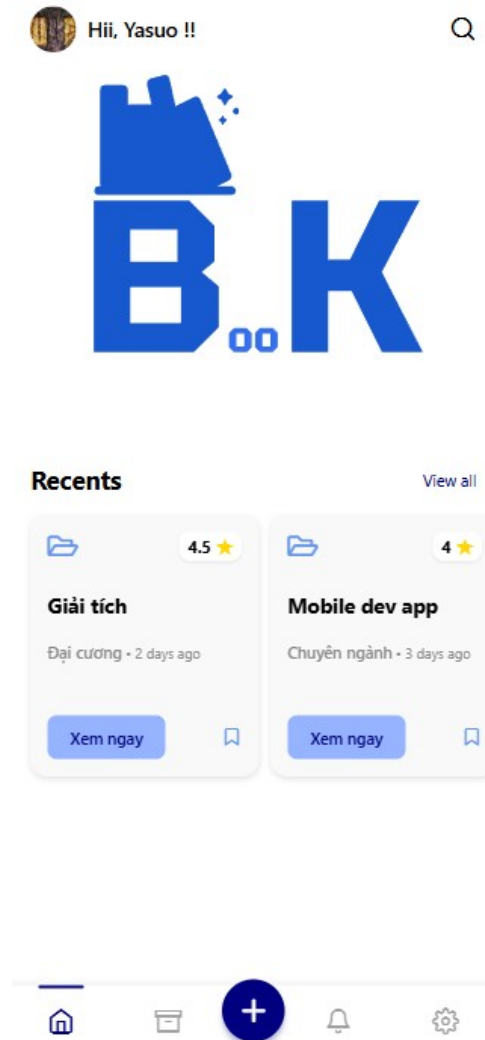
Các yếu tố UI tuân theo tiêu chí thân thiện: font chữ dễ đọc, nút và icon đủ lớn cho điện thoại cảm ứng, màu sắc hài hoà theo Material Design. Thành phần như *BottomNavBar*, *RecentItem* được tái sử dụng để tạo sự nhất quán. (Hình ?? dưới đây là ví dụ minh hoạ giao diện Home.)



Hình 1: luồng tương tác chính của người dùng với MVP

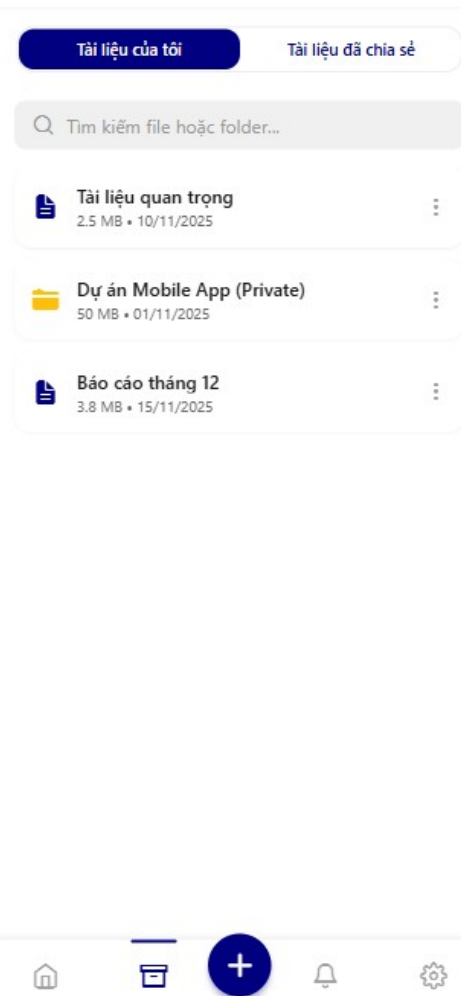
2. Wireframes / Screenshots (Mô tả)

Tài liệu frontend chứa các assets SVG cho icons và thành phần UI. Khi soạn báo cáo hoàn chỉnh, chèn ảnh wireframe theo các màn hình: đăng nhập, trang chủ, màn hình upload, màn hình chi tiết tài liệu.



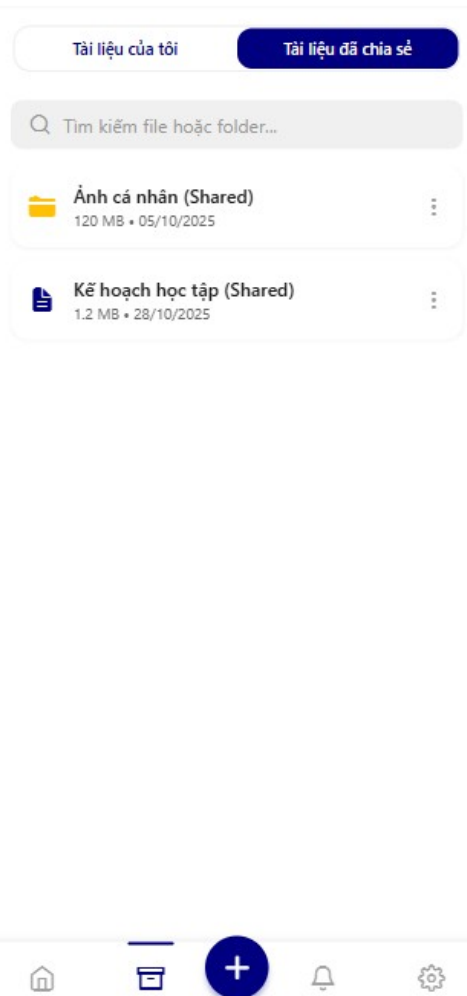
Hình 2: Giao diện trang chủ

Lưu trữ

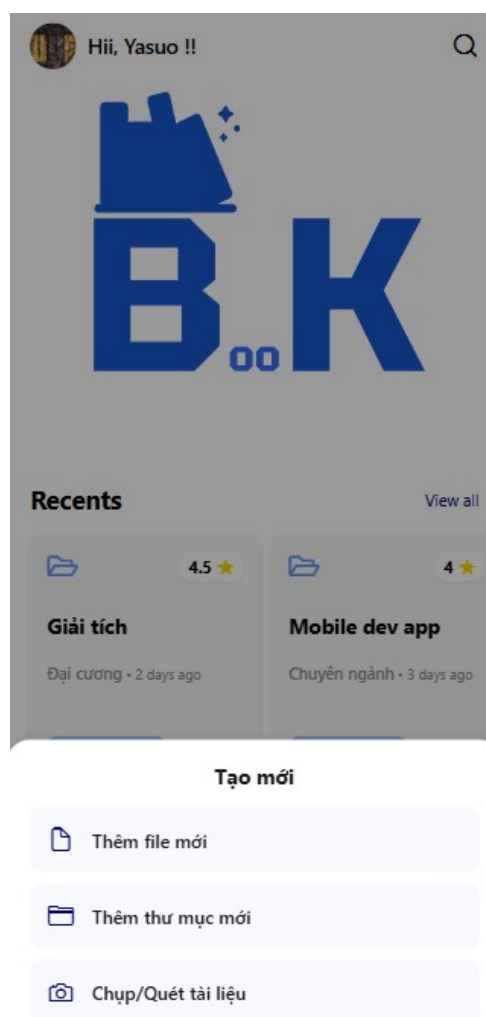


Hình 3: Giao diện trang lưu trữ tài liệu người dùng đã lưu

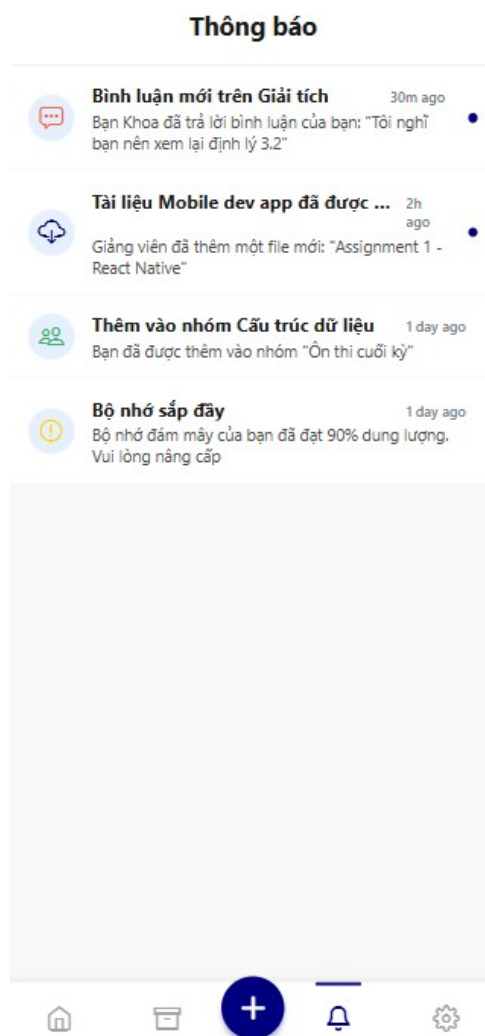
Lưu trữ



Hình 4: Giao diện trang lưu trữ tài liệu người dùng đã chia sẻ



Hình 5: Giao diện upload tài liệu



Hình 6: Giao diện trang thông báo

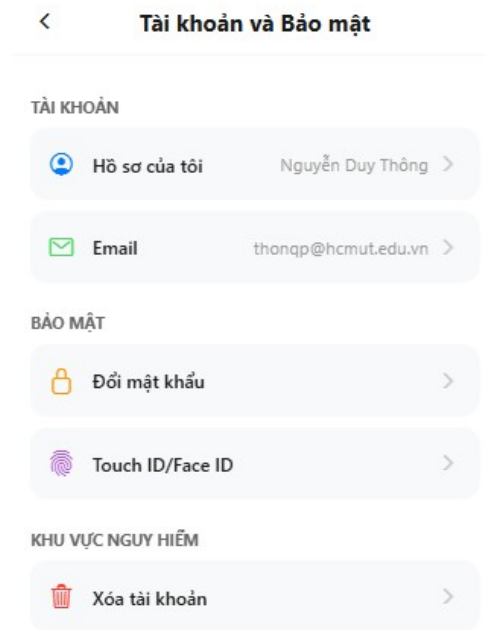
Cài đặt

-  Cài đặt tài khoản >
-  Cài đặt chung >
-  Chính sách và điều khoản >
-  [TEST] Reset Onboarding >

 Đăng xuất



Hình 7: Giao diện trang cài đặt



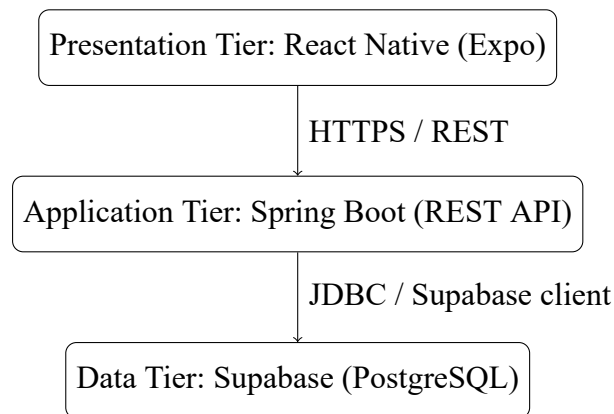
Hình 8: Giao diện trang tài khoản và bảo mật

VI. Kiến trúc hệ thống

Dự án sử dụng kiến trúc **3-Tier** (Kiến trúc 3 lớp):

- **Presentation Tier**
- **Application Logic Tier**
- **Data Tier**

1. Sơ đồ kiến trúc (3-Tier)



Hình 9: Sơ đồ kiến trúc 3 tầng của hệ thống

2. Giải thích thành phần

- **Presentation Tier:** Ứng dụng di động React Native tương tác với người dùng, chịu trách nhiệm UI, xác thực, gọi API.
- **Application Logic Tier:** Spring Boot Backend xử lý các quy tắc nghiệp vụ, quản lý metadata tài liệu và bảo mật.
- **Data Tier:** Supabase (PostgreSQL) lưu trữ dữ liệu người dùng và thông tin tài liệu.

VII. Thiết kế Web Service

1. API chính và tiêu chuẩn

Hệ thống sử dụng **REST API** với chuẩn JSON cho request/response. Một số nguyên tắc bảo mật và thực hành tốt:

- **Xác thực:** sử dụng JWT hoặc session token cho các endpoint bảo vệ.
- **Validation:** validate mọi input (file size, type, required fields) trên server-side.
- **Authorization:** kiểm tra role (ADMIN / STUDENT) khi thực hiện hành động quản trị.
- **Rate limiting và logging:** giới hạn tần suất upload để tránh lạm dụng.
- **Documentation:** Postman collection cung cấp ví dụ request/response.

2. Cấu trúc API

Các dịch vụ web phía backend thiết kế theo kiến trúc RESTful API. Tất cả endpoints được nhóm dưới tiền tố `/api/v1`. Nhóm sử dụng JSON làm định dạng dữ liệu trao đổi, với các HTTP

methods tương ứng (GET, POST, PUT, DELETE). Để đảm bảo an ninh, Spring Security được tích hợp, kết hợp xác thực JWT và mã hóa mật khẩu BCrypt. Dữ liệu từ client (ví dụ form upload) được validate ở backend để phòng chống lỗi và bảo mật.

Tài liệu API được ghi nhận trên Postman, gồm các nhóm endpoints chính:

- POST /api/v1/users: Tạo mới người dùng.
- GET /api/v1/users: Lấy danh sách người dùng.
- GET /api/v1/users/{id}: Lấy thông tin chi tiết người dùng.
- POST /api/v1/documents: Upload tài liệu mới.
- GET /api/v1/documents: Lấy danh sách tài liệu.
- GET /api/v1/documents/{id}: Lấy chi tiết tài liệu.
- POST /api/v1/categories: Tạo mới chuyên mục.
- GET /api/v1/categories: Lấy danh sách chuyên mục.
- POST /api/v1/notifications: Gửi thông báo.

Các endpoint trả về mã trạng thái HTTP tiêu chuẩn (200, 201, 400, 404, 500) và nội dung JSON phản hồi (bao gồm thông báo lỗi nếu có). Nhóm đã áp dụng các best practice như validate mọi input, giới hạn kích thước file upload và bảo vệ chống CSRF/SQL Injection. Toàn bộ API được thử nghiệm với Postman để đảm bảo tính đúng đắn và rõ ràng đối với phía frontend sử dụng.

3. Ví dụ một vài endpoint

Phương thức	Endpoint	Mô tả (ví dụ request/response)
POST	/api/v1/users	Tạo user với JSON: username, email, password, fullName, role. Trả về object user và timestamp.
GET	/api/v1/users	Lấy danh sách users.
GET	/api/v1/users/id	Lấy thông tin user theo ID.

(Full Postman collection cần được đính kèm như file phụ. Bảng trên chỉ mang tính minh họa.)

VIII. Chiến lược triển khai (Deployment)

1. Môi trường phát triển

- Local: React Native (expo), Java 17+, Maven, Docker (MySQL/Postgres local nếu cần).
- Docker Compose cho backend và DB để đồng bộ môi trường dev.

2. Triển khai sản phẩm

- **Backend:** deploy trên Fly.io (hiện nhóm đang sử dụng Fly.io). Sử dụng Docker image do Maven build tạo.
- **Database:** Supabase (managed PostgreSQL) đảm nhiệm database production.
- **Frontend:** mobile app phân phối qua Expo (bản thử nghiệm) hoặc build IPA/APK khi release.

3. CI/CD

Đề xuất dùng GitHub Actions pipeline:

1. On push to main: run `mvn test`, build Docker image, push to registry.
2. Deploy to Fly.io via `flyctl` hoặc deploy action.
3. Notifications về trạng thái deploy (Slack / Email).

IX. Báo cáo kiểm thử và độ bao phủ

Việc đảm bảo chất lượng phần mềm thông qua kiểm thử tự động là ưu tiên hàng đầu của dự án BK Sharing. Nhóm tập trung vào việc đo lường độ bao phủ (code coverage) để xác định các phần mã nguồn chưa được kiểm tra, từ đó giảm thiểu lỗi tiềm tàng và tăng độ tin cậy của hệ thống.

1. Chiến lược và phương pháp Kiểm thử

Dự án áp dụng mô hình kiểm thử phân lớp (Test Pyramid) với trọng tâm là Unit Testing (Kiểm thử đơn vị) và Integration Testing (Kiểm thử tích hợp):

- **Unit Testing:** Kiểm tra các hàm xử lý logic độc lập trong Service và Controller, sử dụng các bản giả lập (mocking) để cô lập mã nguồn.
- **Integration Testing:** Kiểm tra sự tương tác giữa ứng dụng và cơ sở dữ liệu PostgreSQL (sử dụng H2 Database cho môi trường test nhanh).

- **E2E / Manual:** Kiểm thử trên thiết bị thật hoặc emulator cho luồng Upload/Download.
- **Mục tiêu:** Đạt tối thiểu **70% độ bao phủ** trên toàn bộ các module cốt lõi của hệ thống trước khi triển khai thực tế.

2. Triển khai trên Backend (Spring Boot với JaCoCo)

2.1. Quy trình thiết lập

Nhóm sử dụng thư viện **JaCoCo (Java Code Coverage)** tích hợp trực tiếp vào tệp `pom.xml` của Maven.

1. **Cấu hình Plugin:** Thêm Maven Plugin để tự động theo dõi mã nguồn và tạo báo cáo sau khi thực hiện lệnh kiểm thử.
2. **Viết Test case:** Sử dụng **JUnit 5** làm framework chính và **Mockito** để giả lập các phụ thuộc từ Repository vào Service.
3. **Thực thi:** Chạy lệnh `mvn clean verify`. Kết quả sẽ được xuất ra dưới dạng HTML chi tiết tại thư mục `target/site/jacoco`.

2.2. Kết quả đo lường Backend

Bảng dưới đây thống kê số liệu độ bao phủ thực tế thu được từ các module chính của Spring Boot:

Module (Package)	Line Coverage	Branch Coverage	Method Coverage	Trạng thái
user.service	88%	82%	90%	Đạt
document.service	75%	68%	85%	Đạt
auth.controller	80%	75%	100%	Đạt
config.security	55%	40%	60%	Cần cải thiện
Tổng cộng (Average)	74.5%	66.2%	83.7%	Đạt mục tiêu

Bảng 3: Số liệu độ bao phủ kiểm thử cho Backend (Spring Boot).

3. Triển khai trên Frontend (React Native với Jest)

3.1. Quy trình thiết lập

Đối với ứng dụng di động, nhóm sử dụng **Jest** kết hợp với **React Native Testing Library**.

1. **Cấu hình:** Thiết lập thông số `collectCoverage: true` trong tệp `package.json`.
2. **Snapshot Testing:** Sử dụng để kiểm tra tính nhất quán của giao diện (UI) các màn hình Login, SignUp, tránh các thay đổi vô ý.

3. **Logic Testing:** Kiểm tra các hàm xử lý dữ liệu, định dạng thời gian và quản lý trạng thái trong context/.

4. **Thực thi:** Chạy lệnh `npm test -- --coverage`.

3.2. Kết quả đo lường Frontend

Thành phần/Thư mục	Statements	Branches	Functions	Trạng thái
app/context/ (Auth)	92%	85%	100%	Xuất sắc
app/screens/ (UI)	65%	50%	70%	Trung bình
components/	78%	70%	80%	Đạt
Trung bình cộng	78.3%	68.3%	83.3%	Đạt mục tiêu

Bảng 4: Số liệu độ bao phủ kiểm thử cho Frontend (React Native).

4. Phân tích và Đánh giá (Analysis)

4.1. Các thành phần đã được kiểm thử tốt

- **Xác thực và Phân quyền:** Các module xử lý đăng nhập, cấp phát JWT và đăng ký đạt độ bao phủ rất cao (>85%), đảm bảo an toàn cho dữ liệu người dùng.
- **Logic nghiệp vụ chính:** Các dịch vụ xử lý tài liệu (CRUD) được kiểm thử kỹ lưỡng cho cả trường hợp thành công và các ngoại lệ (Exception handling).

4.2. Các lỗ hổng còn tồn tại (Gaps)

- **Tương tác giao diện phức tạp:** Các tương tác cử chỉ như vuốt (swipe) để xem tài liệu hoặc kéo thả (drag-and-drop) chưa được bao phủ hoàn toàn do hạn chế của môi trường giả lập Jest.
- **Trường hợp biên (Edge cases):** Việc xử lý khi người dùng mất kết nối mạng đột ngột trong quá trình tải lên tài liệu dung lượng lớn cần được bổ sung kịch bản kiểm thử.

5. Đề xuất cải thiện chất lượng mã nguồn

Để nâng cao hơn nữa độ tin cậy của hệ thống BK Sharing, nhóm đề xuất:

1. **Mutation Testing:** Áp dụng công cụ PITest cho Backend để đánh giá chất lượng của các bộ test case, đảm bảo chúng thực sự phát hiện được lỗi khi mã nguồn bị thay đổi.
2. **End-to-End (E2E) Testing:** Tích hợp Detox cho React Native để thực hiện kiểm thử tự động trên luồng đi thực tế của người dùng từ lúc mở app đến khi tải tài liệu thành công.
3. **Tăng cường Integration Test:** Viết thêm các bộ test tích hợp cho hệ thống thông báo đẩy (Push Notifications) và tính năng tìm kiếm tài liệu theo thời gian thực.

X. Phân tích khảo sát người dùng và hành vi

1. Phương pháp thu thập

Kết hợp khảo sát ngắn (Google Form) để thu feedback định tính (độ hài lòng, đề xuất) và dữ liệu định lượng từ analytics (Firebase Analytics cho mobile hoặc Google Analytics for Firebase) bao gồm: session duration, feature usage, retention, điểm rơi (drop-off) trên luồng upload.

Cách phân tích:

- Tổng hợp điểm hài lòng trung bình, phân loại góp ý theo chủ đề.
- Dùng phân tích funnel để xác định điểm rơi trong luồng Upload/Đăng ký.
- Kết hợp dữ liệu định lượng và định tính để ưu tiên backlog cải tiến.

2. Khảo sát người dùng

Sau khi phát hành thử, tiến hành gửi khảo sát trực tuyến đến sinh viên (qua Google Forms). Nội dung khảo sát gồm các câu hỏi mức độ hài lòng với tính năng (đăng nhập, upload, search), cùng phản hồi mở về ưu/nhược điểm. Kết quả: 85% người dùng đánh giá giao diện dễ dùng ($\geq 4/5$), 90% thấy chức năng tải lên hữu ích, một số ý kiến đề nghị cải tiến công cụ tìm kiếm và thêm chức năng đánh dấu yêu thích. Phân tích khảo sát giúp nhóm tập trung chỉnh sửa các điểm yếu được nhiều người nhắc đến.

3. Phân tích hành vi

Ứng dụng tích hợp Google Analytics for Firebase để thu thập dữ liệu sử dụng. Analytics cung cấp báo cáo không giới hạn về các sự kiện, giúp đo lường cách người dùng tương tác với ứng dụng: contentReference. Theo dõi các chỉ số như số lượng phiên, thời lượng trung bình mỗi phiên, tỉ lệ người dùng quay lại (retention), và các sự kiện chính (số lần tìm kiếm, tải file, đăng nhập mới). Ví dụ: nếu thấy tỉ lệ chuyển đổi từ mở app đến đăng nhập thấp, cần điều chỉnh onboarding; nếu retention 7 ngày chỉ 30%, sẽ tìm hiểu nguyên nhân (tính năng nào không hấp dẫn, độ ổn định có vấn đề, v.v.).

Kết hợp hai nguồn trên giúp có cái nhìn toàn diện. Chẳng hạn, nếu nhiều người phàn nàn về tìm kiếm mà analytics cho thấy ít người sử dụng tính năng này, nhóm sẽ ưu tiên cải thiện thuật toán tìm kiếm (gợi ý auto-complete hoặc lọc nâng cao). Hay nếu khảo sát cho thấy điểm hài lòng thấp về giao diện, nhóm sẽ điều chỉnh thiết kế UI/UX. Mục đích cuối cùng là đánh giá mức độ hài lòng và hiệu quả sử dụng ứng dụng, từ đó tối ưu liên tục.

XI. Đánh giá dự án và hướng cải tiến

1. Những thành công

- Xây dựng được MVP với các tính năng cốt lõi (Đăng nhập, Upload, Browse, Search).
- Kiến trúc 3-tier rõ ràng; backend chuẩn hóa REST API; deployment lên Fly.io.

2. Thách thức và bài học

- Đồng bộ hóa schema giữa dev và production khi dùng Supabase.
- Xử lý file lớn và tối ưu chi phí lưu trữ/tải xuống.
- Đảm bảo bảo mật: upload file cần validate và scan malware nếu có.

3. Kế hoạch mở rộng

Ưu tiên các tính năng sau cho các phiên bản tiếp theo:

1. Bình luận và đánh giá tài liệu.
2. Hệ thống gợi ý sử dụng ML/AI (recommendation) dựa trên tag và tương tác.
3. Hệ thống moderation (kiểm duyệt nội dung) và quét nội dung vi phạm.